KSOLVES
AI First, Always

En(AI)bling™

Agentic AI Hackthon 2026

Build an AI Agent That Actually Ships

A challenge in Agentic AI engineering. Not a demo. Not a toy. Something real.

3 days	30 pts	20	100%
Time	Production readiness	Mock tickets	AI tools allowed

AI First, Always

The Era of Autonomy Begins.

Stop talking to AI. Start building with AI.

2,000+

Developers Joined from across the globe

"The world has enough chatbots."
We are here to build agents.

3

DAYS

GLOBAL AGENTIC AI
2026

Your Mission next 3 days

Don't write code. Engineer Reasoning.

1
Solves Real Friction

We aren't building toys. Focus on the pain points of 2026 that agentic architecture can eliminate - autonomously and at scale.

2
Proof of Work

Your submission by Sunday IS your resume. It is your ticket into the world's most AI-first companies leading architecture across industries.

3
The Leaderboard

Hiring partners and mentors are watching closely. They aren't looking for AI hobbyists - they're looking for the next AI Architects.

PROBLEM | Autonomous Support Resolution Agent

ShopWave receives hundreds of support tickets daily. Most are repetitive - but all go to humans. Your job: build an agent that resolves them autonomously. (Sample Data: https://github.com/ksolves/agentic_ai_hackthon_2026_sample_data/)

01 Ingest

Tickets from simulated source

02 Classify & Triage

By urgency, category, and resolvability

03 Autonomously Resolve

Use tools to take action - don't just classify and stop

04 Escalate Intelligently

When uncertain, hand off with a structured summary

05 Audit Every Decision

Log tool calls, reasoning, and outcomes - not just final answers

CONSTRAINTS |

Rules Your Agent Must Follow

Chain

Must make at least 3 tool calls in a single reasoning chain. E.g., lookup user → check order → send reply.

Recover

Must gracefully handle tool failures - at least one tool will timeout or return malformed data. Don't crash.

Concurrency

Tickets must be processed concurrently, not one-by-one. Sequential processing will be penalised.

Explain

Every decision must be explainable. No black-box outputs.

AI coding tools are fully allowed - but you will be asked to walk through your code live. If you can't explain it, it counts against you.

TOOLS | You Implement the Mocks. We Evaluate How.

READ / LOOKUP

get_order(order_id)
Order details, status, timestamps

get_customer(email)
Customer profile, tier, history

get_product(product_id)
Product metadata, category, warranty

search_knowledge_base(query)
Policy & FAQ semantic search

WRITE / ACT

check_refund_eligibility(order_id)
Returns eligibility + reason. May throw errors.

issue_refund(order_id, amount)
IRREVERSIBLE - must check eligibility first

send_reply(ticket_id, message)
Sends response to the customer

escalate(ticket_id, summary, priority)
Routes to human with full context

Lazy mocks = lazy thinking. Build mocks that fail realistically - timeouts, malformed data, partial responses.

RECOMMENDED STACK & TIPS

Use what you know. Build what works.

STACK OPTIONS

Orchestration

LangGraph, AutoGen, or custom ReAct loop

LLMs

OpenAI, Anthropic, Gemini, or Ollama (open-source)

Language

Python preferred — TypeScript accepted

Infra

Local-first is fine. Docker = bonus points.

WHAT SEPARATES GOOD FROM GREAT

Retry budgets

Backoff on tool failures, not silent errors

Dead-letter queues

Failed tickets don't disappear — they get logged

Confidence calibration

Your agent should know what it doesn't know

Schema validation

Validate tool outputs before acting on them

Good luck. Build something real.

The best submission will make us ask — why isn't this already in production?

DELIVERABLES |

Submission Checklist

01

Working Agent

Runnable code with a clear entry point. README is mandatory.

02

Architecture Diagram

1-page: agent loop, tool design, memory/state management

03

Failure Mode Analysis

At least 3 failure scenarios documented with how your system responds

04

Demo Run

Live or recorded — processing all 20 mock tickets end-to-end

KSOLVES
AI First, Always

En(AI)bling™

Submission Instructions (Deadline - Sunday, 19th April, 9 PM IST)

Step 1 - Push your code to GitHub

Create a public GitHub repository named hackathon2026-[your-name]. Make sure your entire project is pushed before the deadline. Commits after the deadline will not be considered.

Step 2 — Ensure your repo includes the following:

- README.md - Setup instructions, how to run the agent, and your tech stack. Missing README = automatic 10 point deduction.
- architecture.pdf or architecture.png - Your 1-page agent loop and tool design diagram.
- failure_modes.md - At least 3 documented failure scenarios and how your system handles them.
- audit_log.json or audit_log.txt - Generated output from your demo run showing tool calls, reasoning, and decisions for all 20 tickets.
- Recorded Demo

Step 3 - Record or prepare your demo

Run your agent against the provided data and either record a short screen recording (MP4, max 5 mins) or be ready to run it live during judging. Upload the recording to the repo.

Step 4 - Check before you submit

- No hardcoded API keys or secrets anywhere in the repo
  Your agent runs with a single command (document it in the README)
- All 5 deliverables are present and named correctly
- Your audit log covers all 20 tickets

Step 5 - Submit the form

Fill out the submission form at [https://forms.gle/dXkrnQr7vDkxraqZ7] with:

- Your name
- GitHub repo link
- One line on the biggest challenge you faced

JUDGING | 100 Points Total

30 pts

Production Readiness

Modularity, error handling, logging, security

30%

10 pts

Agentic Design & Thoughtfulness

Agent loop quality, tool design, when NOT to act

10%

30 pts

Engineering Depth

Concurrency, mock quality, state management, code quality

30%

10 pts

Evaluation & Self-awareness

Confidence scoring, failure awareness, feedback loop

10%

20 pts

Presentation, Deployment

Explain every decision. Live Application

20%

RULES |

Don't Get Tripped Up

- **FATAL** No working demo → not evaluated at all
- **FATAL** Agent only does single-step LLM calls with no real tool use → disqualified as agentic
- **FATAL** Hardcoded API keys anywhere in the repo — immediate security flag
- **FATAL** README missing from submitted repo and architecture documents

These aren't tricks. They reflect real engineering standards. Treat this submission like it's going to production.

All the Best!

Thank you for participating.